Final Project Report
(50.039 Deep Learning, Y2026)

Submitted by:
Keagan Teo (1007779)
Cheng Mun Yun Valerie (1007879)

Team Members and Contributions

Our team consists of two members:

Name	Student ID	Contributions
Keagan Teo	1007779	Feature engineering, model tuning, report writing
Cheng Mun Yun Valerie	1007879	Model implementation, data visualisations, report writing

Topic and Machine Learning Problem

The topic that we have chosen to investigate is the detection of anomalous network traffic. As students specializing in cybersecurity, we find that this topic is the most suitable and interesting for us. Not only that, identifying and blocking suspicious network traffic has always been pivotal in the world of cybersecurity. With companies facing hundreds of thousands of network packets on a daily basis, and with threat actors utilizing AI to further complicate their attacks and attack patterns, it has become harder than ever to detect malicious activity within a network. We hope to leverage deep learning to detect such malicious activity.

The machine learning problem is a supervised classification task to predict whether a particular network packet is malicious or benign.

Dataset

The dataset that we used can be found here. We chose this dataset primarily for its heavy class imbalance, with significantly more benign classes than malicious ones.

We have chosen to use only one of the files within the dataset, the one named “UNSW-NB15_1.csv”, as using all 4 files would lead to an extremely long training time. This file already has around 70,000 samples and we believe it is sufficient to train a well performing model. It is important to note that we did not use the training and test sets that came with the dataset as after inspecting the data within those sets, we found that there were no class imbalances within them, and would hence violate the requirement of this project.

Preparation of Data

Dropping Features

After evaluating our dataset, we found some features that were not useful in the determination of a malicious packet and have dropped them. The features are:

- "srcip" and "dstip" are raw ip addresses that won't generalize across datasets. Behavioral patterns from IPs are already captured by the ct_features

- "stime" and "ltime" are absolute timestamps of the packet and carries no signal on whether traffic is malicious
- "stcpb" and "dtcpb" source and destination TCP base sequence number is randomly initialized and does not help with categorization
- "sport" and "dsport" are raw port numbers and service and protocol that can be deduced from port numbers are already captured in "proto" and "service" features
- "attack_cat" is an extension of a label and should not be included in the training data

One-Hot Encoding

Since the neural network only accepts numerical data, we would have to somehow represent non-numerical columns numerically. We have chosen one-hot encoding as the method to do so. By creating a column for each unique value (or groups of values as we would explore later on) within a column, and indicating with a 1 or 0 for each entry the corresponding value, we would have created a numerical representation of the non-numerical data.

While processing the data, we encountered a problem: there were too many unique values for certain columns, leading to a huge increase in the number of columns that we have, with many values being 0. This would lead to the dataset being very large and sparse, leading to slower training times and possibly memory issues. Our fix is to group values into categories where possible for columns with high cardinality.

Normalizing

Normalizing columns is essential to ensure that our model trains and performs well. Values that are too large or too small might lead to exploding or vanishing gradients. Additionally, features with larger magnitudes can dominate the model's training simply because they are larger. We have implemented a few different normalization techniques for our dataset, depending on the feature's nature and distribution.

- Log + StandardScaler: for columns which have skewed values
- StandardScaler: for columns that already follow a roughly normal distribution
- MinMaxScaler: for columns that have known bounded values

Deep Learning Architecture

We had originally planned to use 4 Linear + ReLU layers, but have since decided to use 4 Linear + BatchNorm + ReLU + Dropout layers instead, incorporating BatchNorm and Dropout functions into each of our hidden layers.

Additionally, after processing the dataset and conducting feature engineering, the total number of input features turned out to be 73. Our final deep learning architecture is as follows:

Input (73 features)

↓

Linear (256) + BatchNorm + ReLU + Dropout

↓

Linear (128) + BatchNorm + ReLU + Dropout

↓

Linear (64) + BatchNorm + ReLU + Dropout

↓

Linear (32) + BatchNorm + ReLU + Dropout

↓

Linear (1) + Sigmoid

Loss Function

Addition of Weight

Since our dataset is heavily skewed, with many more benign network packets than malicious ones, we had to adapt our binary cross entropy loss function to account for this. We added a weight to the loss function for malicious packets, penalizing the misclassification of a malicious packet much more than a misclassification of a benign one. The weight has been defined as:

$$w_i = \begin{cases} pos_weight \;\; if \;\; y_i{=}1 \;\; (malicious) \\ 1.0 \;\; if \;\; y_i{=}0 \; (benign) \end{cases}$$

$$pos_weight = \frac{N_{benign}}{N_{malicious}}$$

This solves the problem of the skewed dataset, with the malicious class being emphasized a lot more, proportional to the ratio of number of benign classes to the number of malicious classes. Moreover, this addition also works in our favour as a misclassification of a malicious packet has far greater cost than a misclassification of a benign packet in the cybersecurity context. The weight added to packets that have true labels as malicious ensures that false negatives are heavily penalized.

Final Loss Function

Our final loss function is therefore mathematically defined as:

$$L = -\frac{1}{N}\sum_{i=1}^{N} w_i[y_i log(\widehat{y_i}) + (1 - y_i)log(1 - \widehat{y_i})]$$

Optimizer and Scheduler

We have chosen Adam as our optimizer, and we use it with a L2 weight decay. Additionally, we have introduced a scheduler to reduce the optimizer's learning rate when the validation loss plateaus, in hopes of coercing the algorithm to the minima.

Training Configurations

Initialization

We utilized a He uniform initialization for our hidden layers.

Early Stopping

We have implemented early stopping with patience, which defines the consecutive number of epochs with no improvement to validation loss. Once this patience number is hit, we simply stop the model from training, justifying that any further training will lead to overfitting on the training data.

Saving the Model

At each epoch, we will conduct a check on the validation loss, and if the current validation loss is lower than the lowest so far, we will keep the model weights in memory to be then reloaded back after training.

Train-Validation-Test Split

We utilized a train-validation-test split of our data set with a ratio of 0.8:0.1:0.1 respectively. Stratify was used to ensure that the splits will maintain roughly equal proportions of benign and malicious packets. This is especially important with anomaly detection as we would not want malicious packets to be underrepresented in any of the splits.

Evaluation of Model

Evaluation Metrics

We have chosen a couple of metrics to evaluate the performance of our model with. By using a combination of these metrics, it paints us a fuller picture of how well our model is actually performing.

Loss

Will tell us if the model is training properly; whether the model is overfitting or underfitting. The loss function is defined above.

Accuracy

The percentage of all predictions the model got right. This metric tells us how often the model is right overall.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Confusion matrix

A breakdown of exactly where the model is right and wrong across all four possible outcomes, telling us exactly where the model is succeeding and failing.

$$Confusion\ matrix = \begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}$$

Precision

Of all the packets flagged as malicious, how many actually were. Indicates how trustworthy a positive result by the model is.

$$Precision = \frac{TP}{TP + FP}$$

Recall

Of all the actual malicious packets, how many the model caught. Indicator of how well the model is at not missing malicious packets.

$$Recall = \frac{TP}{TP + FN}$$

F1

A single score balancing precision and recall. Indicates how well the model is at balancing between catching attacks and avoiding false alarms.

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$$

ROC-AUC

How well the model separates malicious from benign across every possible threshold. Indicates how well the model fundamentally distinguishes malicious from benign traffic regardless of threshold.

The ROC curve plots TPR (True Positive Rate) against FPR (False Positive Rate) at every threshold $t \in [0, 1]$

$$TPR(t) = \frac{TP(t)}{TP(t) + FN(t)}$$

$$FPR(t) = \frac{FP(t)}{FP(t) + TN(t)}$$

AUC is the area under that curve:

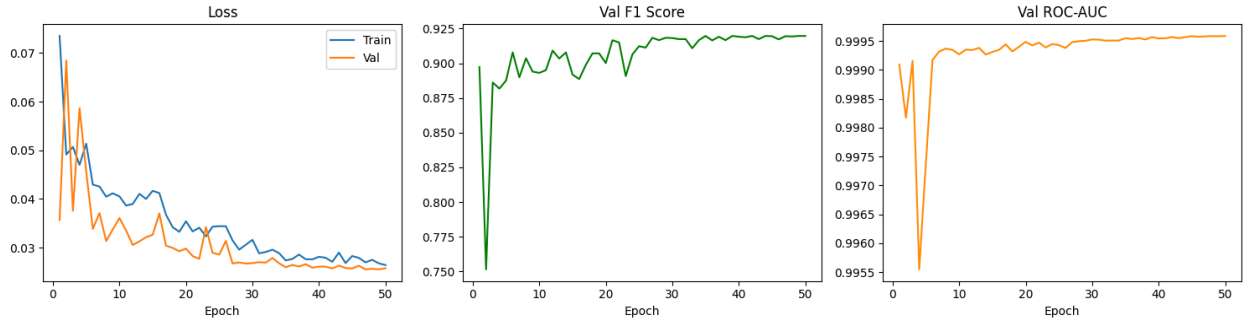
$$AUC = \int_0^1 TPR(t) d(FPR(t))$$

Evaluation

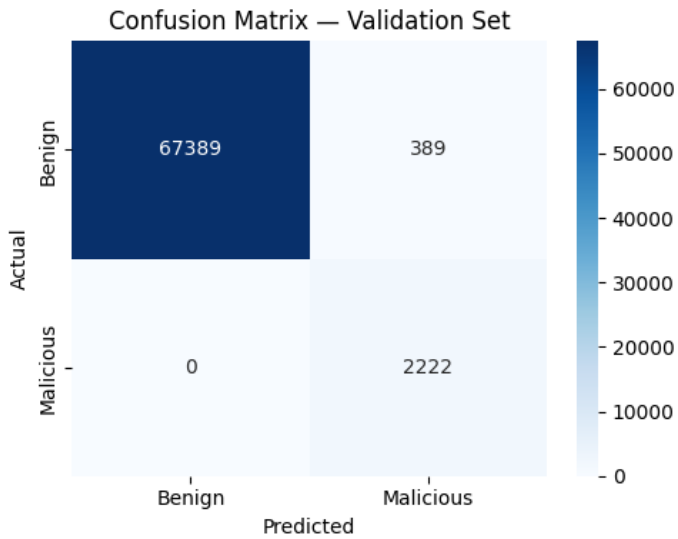
After training the model that we have so far, taking the best model with the lowest validation loss and obtaining our evaluation metrics with the validation set, we obtain the following results:

```
=== Validation Set Results ===
Loss      : 0.0255
Accuracy  : 0.9944
Precision : 0.8510
Recall    : 1.0000
F1        : 0.9195
ROC-AUC   : 0.9996
```

We obtained the following training curves:



And the following confusion matrix:



Taking a look through the results, we see that our model has trained well and is performing quite well. Looking at the loss curve, we notice that there may be a slight overfitting on the training data towards the end, but due to our early stopping and model saving, we ensure that only the best model gets saved and loaded after training ends.

Recall is at 1.0, meaning that all malicious packets were classified correctly, and is reflected in the confusion matrix as well. This means that there were no false negatives, and is essential in our use case as any malicious packets that do not get detected could lead to disastrous results.

The other metrics are also at very good values, RPC-AUC, F1 and Accuracy are all very close to 1.

Tuning the Model

The model is already performing really well, so the goal of tuning would mainly be to reduce the number of false positives. We will be using Optuna, a hyperparameter tuning framework to help find the best hyperparameters for our model.

We have a few parameters we would like to tune:

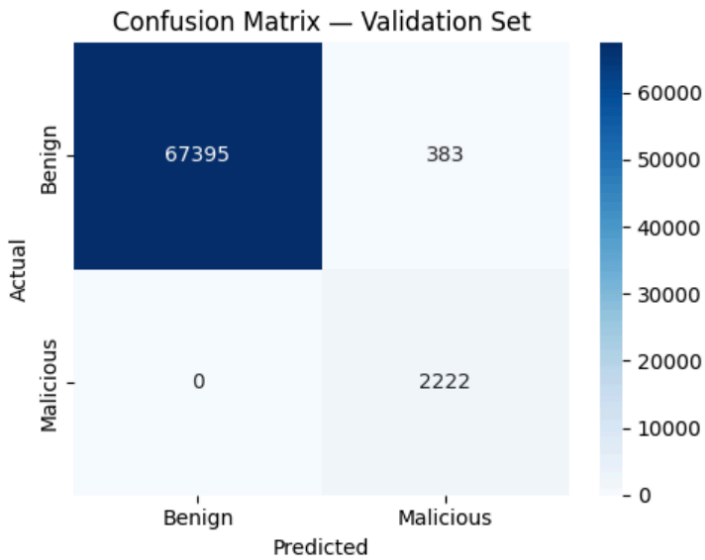
- Learning rate
- Weight decay
- Dropout probability
- Batch size
- Scheduler factor
- Scheduler patience

After running the tuning.py file, we get a set of best hyperparameters and the saved weights for those hyperparameters:

```
============================================================
Best trial: #25
  val_loss : 0.022459831493241447
  val_f1   : 0.9206546509218977
  Params:
    lr: 0.000503214096782759
    weight_decay: 2.1258108517939597e-06
    dropout_p: 0.12296297251821234
    batch_size: 512
    scheduler_factor: 0.5069410260856266
    scheduler_patience: 3

Best params saved to  : tuning/best_params.json
Best model weights    : tuning/best_model.pth
```

Visualising the best model's performance on the validation set:



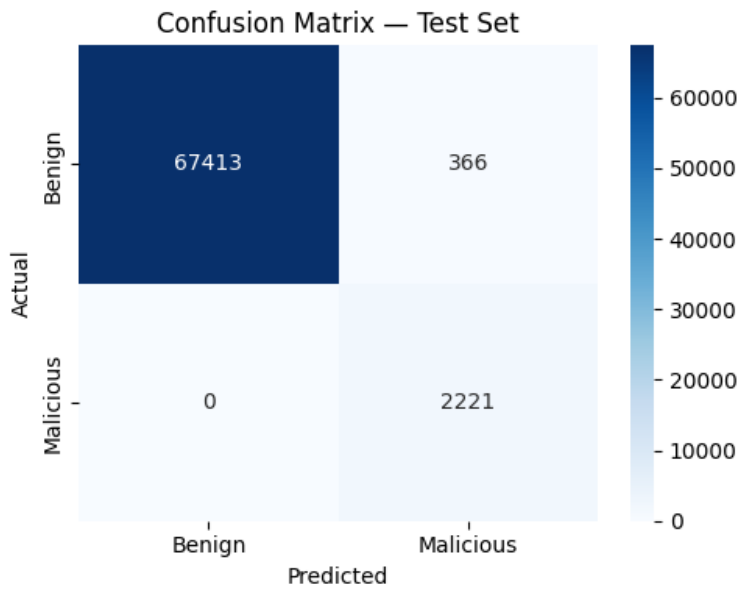
We see that the number of false positives have barely dropped, from 389 previously to 383 after the hyperparameter tuning. This is an indication that the model was already performing very well and the hyperparameters that we started out with were already pretty good.

Final Evaluation on Test Set

We conduct our final evaluations on the test set and achieved the following results:

```
=== Test Set Results ===
Loss      : 0.0224
Accuracy  : 0.9948
Precision : 0.8585
Recall    : 1.0000
F1        : 0.9239
ROC-AUC   : 0.9998
```

Visualising with a confusion matrix:



We see that the model has trained and generalized relatively well.

Comparison with State of the Art Models

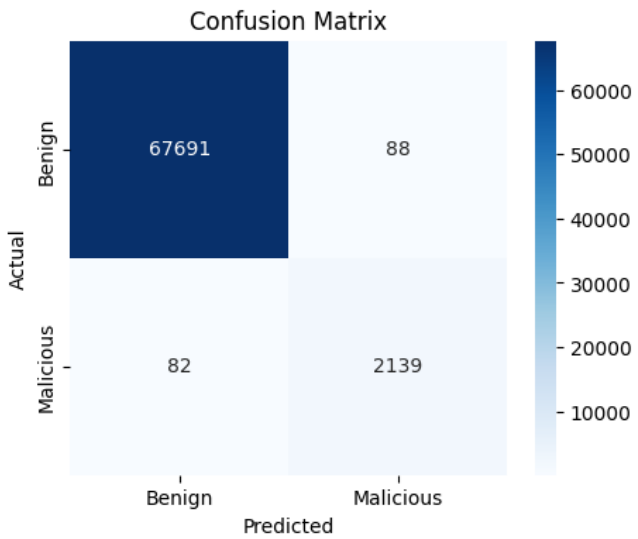
XGBoost

XGBoost (Extreme Gradient Boosting) is a machine learning algorithm based on gradient boosted decision trees, and is frequently the top classical model on IDS benchmarks.

After fitting the model with the training data and evaluating it on the test set, we get the following results:

```
Loss      : 0.0078
Accuracy  : 0.9976
Precision : 0.9605
Recall    : 0.9631
F1        : 0.9618
ROC-AUC   : 0.9997
```

And the following confusion matrix:



We see that the model has much fewer false positives, but also a greater number of false negatives, which in our use case, is detrimental. However, this may not be a fair comparison as we are not using the same loss functions in the comparison. The loss function used for XGBoost is a simple log loss function, without the weights introduced for our model's loss function.

Running the Model

Installing Dependencies

The list of dependencies are described in the requirements.txt file. To install the dependencies, run the following commands within the project folder:

```
$ python3 -m venv venv
$ source venv/bin/activate
$ pip install -r requirements.txt
```

Since the PyTorch installation depends on your compute platform (cuda version/cpu), you would have to install PyTorch manually. [This link]() will provide you with information on how to install PyTorch.

Downloading the Dataset

Simply run the download_dataset.py setup file:

```
$ python3 download_dataset.py
```

Training the Model from Scratch

To train the model from scratch, simply run all the cells within the project.ipynb file. The way we interacted with the ipynb file was through the Jupyter VS Code extension, but you may use any methods to do so.

To run the hyperparameter tuning from scratch, first run the tuning.py file, then run all cells within the project.ipynb file.

Reloading the Model

To reload a pretrained model, run all the cells within the project.ipynb file except for cells within the “Training the Model” section. Below are the screenshots of the cells you **should not run** to reload a pretrained model.

Training the Model

! Skip this part if you want to reload a trained model's weights

```
history, model = train(model, optimizer, scheduler, train_loader, val_loader, loss_func, DEVICE, NUM_EPOCHS, PATIENCE, verbose=True)
```

✓ 14m 30.4s

Saving the model

```
torch.save(model.state_dict(), 'saved_models/model_before_tuning.pth')
```

✓ 0.0s

By omitting the running of these two cells, the model that we have trained will be used and the images shown in this report for the performance and evaluation of the model can be verified.